

Frontier:

set of nodes available for expansion.  
The set of states that we can transition to based upon applying each and every action to the current state.

Solution:

path from initial state to goal state.

State spaces grow exponentially when:  
many actions per state  
deep search trees.

#### 11. Abstraction in Search

Real world is extremely complex.

Search requires abstraction:

Abstract state:

represents many real-world configurations.

Abstract action:

represents many physical actions.

Example: Arad → Zerind

represents multiple actual driving routes.

Abstraction is valid if:

the abstract solution corresponds to a real-world solution.

#### 12. Tree Search Framework

General algorithm:

Initialize frontier with initial state

Loop:

if frontier empty → failure

choose node to expand  
(strategy-dependent)

if node contains goal → return solution  
expand node and add successors to frontier

Search strategy determines:

which node to expand next  
performance of algorithm.

General problem in Tree Search:  
repeated visiting of same states,  
resulting in higher time complexity

Graph search: uses an auxiliary data structure to record the list of already visited states. If a potential child node state has already been visited, don't visit it again as it has already been visited.

Higher space complexity of graph search than tree search.

#### 13. Nodes vs States

State:

representation of a configuration.

Node:

structure used in search tree.

Node contains:

state

parent node

children

action

path cost

depth

Important: States do NOT store parent or depth — nodes do.

#### 14. Search Algorithms (Overview)

Search algorithms differ by:

node expansion order

memory usage

guarantees

Two major classes:

Uninformed search:

no domain knowledge

"Blind search algorithm"

Uses only information available in problem definition

Doesn't use any external knowledge about where the goal is located.

Informed search:

uses heuristics

This section sets foundation for BFS, DFS, UCS, A\*.

#### 15. Uninformed Search (Intro)

Strategies that use only:

initial state

successor function

goal test

path cost

No heuristics.

Examples:

breadth-first search: Exploring all nodes in a tree or graph level by level.

Implemented as a FIFO Queue. Does BFS guarantee returning the shortest path to a solution? Yes, if the cost of every action is equivalent. All paths could cost only 1.

depth-first search: Frontier is a LIFO QUEUE.

#### 18. Depth-Limited Search (DLS)

Depth-Limited Search is a version of DFS with a maximum depth limit.

Instead of exploring indefinitely, the search:

stops expanding nodes after a fixed depth L

prevents infinite descent in infinite state spaces

Think of it as: DFS + safety boundary.

Why it exists

Regular DFS has a major problem:

can go infinitely deep

may never find a solution even if one exists at a shallow level

Depth-Limited Search avoids this by enforcing a cutoff.

Behavior

If solution depth  $\leq$  limit: → DLS can find solution.

If solution depth  $>$  limit: → DLS fails (cutoff occurs).

Completeness

Not complete unless:

limit  $\geq$  depth of shallowest goal

Optimality

Not optimal.

Why?

deeper, cheaper paths may be ignored

DFS nature does not guarantee shortest path

Time Complexity

$O(b^l)$

b = branching factor l = depth limit

Space Complexity

$O(b \cdot l)$

Very memory efficient (like DFS).

Possible outcomes

DLS returns one of three:

solution

cutoff

failure

Cutoff means: → search stopped due to depth limit, not because no solution exists.

#### 19. Iterative Deepening Search (IDS)

Iterative Deepening Search combines:

DFS space efficiency

BFS completeness

It repeatedly runs Depth-Limited Search with increasing limits.

Depth sequence:  $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$  until goal found.

Idea

Instead of committing to one deep DFS path:

Search shallow levels first, then gradually explore deeper.

This mimics BFS ordering.

Why it works well

Most nodes in a search tree are near the bottom.

Repeatedly exploring shallow nodes:

adds small overhead

but gains completeness and optimality.

Completeness

Yes.

Guaranteed if:

branching factor finite

Optimality

Yes (for unit step costs).

Because:

shallowest goal found first (like BFS).

Time Complexity

$O(b^d)$

Same as BFS.

Even though nodes are re-expanded, the overhead is small.

Space Complexity

$O(b \cdot d)$

Same memory use as DFS.

This is the BIG advantage over BFS.

Why IDS is important

It gives:

BFS optimality

DFS memory usage

This makes IDS one of the most practical uninformed search strategies.

#### 20. DLS vs IDS (High-yield comparison)

#### 21. Common Exam Traps

Professors love these:

Trap 1 "Is DFS complete?" → No (in infinite depth spaces).

Trap 2 "Is DLS complete?" → Only if limit  $\geq$  goal depth.

Trap 3 "Is IDS inefficient because it repeats work?" → No — repeated work is small vs benefits.

Trap 4 "Which algorithm is best when memory is limited but optimal solution required?" → Iterative Deepening Search.

22. When to Use Each

Use Depth-Limited when:

you know max solution depth

you want to prevent infinite DFS

Use Iterative Deepening when:

depth unknown

memory limited

want optimal shallow solution

This is why IDS is often preferred over BFS in practice.

Module 3

1. Why Informed Search Exists

Uninformed search algorithms (BFS, DFS, UCS):

use no knowledge about the goal

explore blindly

often expand huge numbers of nodes

Informed search introduces heuristics to guide exploration and gather knowledge about the remaining distance to a goal.

Core idea:

Use knowledge about the problem to prioritize promising paths.

Instead of exploring all nodes equally, informed search focuses on:

states likely closer to the goal.

This dramatically improves efficiency.

Search algorithms based on an evaluation function

Straight line distance: what's the straight line distance, ignoring any intermediate points between each of the other points head exactly in the direction of the destination without having to worry about any obstacles, cities, roads, etc. in your path. Usually an underestimate.

Does not guarantee to always find the optimal solution

Ex: greedy search is guaranteed to find solutions faster than uninformed search algorithms but is still not guaranteed to be optimal.

Uniform cost search

A tree search algorithm used for traversing or searching a weighted tree, tree structure, or graph. (Type of uniformed search algorithm)

Uses a Priority Queue

Expand node with lowest path costs

Frontier = priority queue ordered by  $g(n)$

Problem with UCS: can expand in unnecessary opposite directions of the

goal state as it is still uninformed.

Path cost: how much does it cost to reach a paritruclar node from the intial node on our search tree

2. Heuristics: The Core Idea

A heuristic is:

a rule of thumb

an educated guess

an estimate that guides search

A heuristic function:

$h(n)$  = estimated cost from state  $n$  → goal.

Key properties:

not required to be exact

useful even when imperfect

speeds up search dramatically

Heuristic knowledge reduces:

search depth

number of explored nodes.

3. Best-First Search Framework

General approach:

Select the node with the lowest evaluation score:

$f(n)$

Frontier:

priority queue sorted by  $f(n)$

Different algorithms depend on: → how  $f(n)$  is defined.

This creates:

greedy search

A\* search

uniform cost search (special case).

4. Evaluation Functions

Search is guided using:

$f(n)$  = estimated total cost of solution through  $n$ .

Key components:

$g(n)$  = cost from start →  $n$   $h(n)$  =

estimated cost from  $n$  → goal

Different search algorithms:

Uniform cost:  $f(n) = g(n)$

Greedy best-first:  $f(n) = h(n)$

A\*:  $f(n) = g(n) + h(n)$

This is a major exam concept.

5. Greedy Best-First Search

Greedy search expands: → node that appears closest to goal.

Uses:  $f(n) = h(n)$

Focus:

ignores path cost so far

only cares about estimated distance to goal

Behavior

Fast and focused, but risky.

May:

get stuck

take long detours

produce non-optimal solutions

Completeness

No (can get trapped).

Optimality

No.

Strength

extremely fast in many problems.

Weakness

poor reliability

ignores real costs.

6. Example Heuristic

Straight-line distance:

common heuristic in map problems.

Used in:

Romania pathfinding example.

It estimates: distance to goal ignoring roads.

This provides: reasonable approximation.

7. A\* Search — The Most Important Algorithm

A\* combines:

real cost so far

estimated remaining cost

Formula:

$f(n) = g(n) + h(n)$

Interpretation:

$g(n)$ :

how far we've come.

$h(n)$ :

how far we estimate remains.

A\* chooses nodes with: → lowest estimated total solution cost.

8. Why A\* Works So Well

It balances:

exploration vs exploitation.

Greedy search:

focuses only on  $h(n)$

Uniform cost:

focuses only on  $g(n)$

A\*:

considers both.

Result:

finds optimal solutions

with far fewer expansions than UCS.

9. Admissible Heuristics

A heuristic is admissible if:

$0 \leq h(n) \leq$  true cost to goal.

Meaning:

never overestimates.

Also called: optimistic heuristic.

Guarantee: A\* with admissible heuristic → optimal solution.

This is one of the most tested exam theorems.

20. Why A\* Is Considered the "Gold Standard"

A\* is one of the most important search algorithms because it:

expands promising paths first

avoids exploring clearly expensive paths

guarantees optimal solutions when the heuristic is admissible

It is a best-first search algorithm where:

$f(n) = g(n) + h(n)$

This balances:

cost so far

estimated remaining cost

This is why it performs better than:

uniform-cost search (too cautious)

greedy search (too risky)

## 21. Frontier Behavior in A\*

A\* uses:

priority queue frontier

ordered by smallest  $f(n)$

At each step: → expand node with lowest estimated total solution cost.

Important detail: Even if a goal appears in the frontier: → DO NOT immediately return it.

You must: → confirm no lower-cost path exists first.

## 22. Why Admissibility Matters (Intuition)

Admissible heuristic:  $h(n) \leq$  true remaining cost.

Meaning:

optimistic

never overestimates

Why this guarantees optimality:

If a heuristic overestimates: → the optimal path may appear worse → gets delayed in expansion → algorithm may return suboptimal goal first.

If heuristic underestimates: → optimal path always stays competitive in frontier ordering.

Therefore: A\* expands nodes in correct cost order.

## 23. Proof Sketch Idea for A\* Optimality (Conceptual)

Key idea:

Let: A = optimal goal B = suboptimal goal

We want: A expanded before B.

Because:

$f(n)$  for ancestors of A  $\leq$  true cost of A and:

$\text{true cost}(A) < \text{true cost}(B)$

Therefore:

$f(A) < f(B)$

Since A\* always expands lowest  $f(n)$ : → ancestors of A expand first → A reached before B

Therefore: A\* finds optimal solution first.

You don't need full proof on exam, but you must understand reasoning.

## 24. Heuristic Design via Relaxed Problems

Powerful exam concept.

To create admissible heuristic:

Remove constraints from original problem

Solve relaxed version

Use cost as heuristic

Why it works: Relaxed problem cost  $\leq$  real problem cost.

Examples (8-puzzle):

Original constraints:

move only into adjacent blank

Relaxations:

Relaxation 1:

move tile to any adjacent square → Manhattan distance

Relaxation 2:

move tile anywhere → tiles out of place

Both:

admissible

underestimate real cost.

## 25. Dominance of Heuristics

Heuristic  $h_1$  dominates  $h_2$  if:

for every node:  $h_1(n) \geq h_2(n)$

AND both admissible.

Dominating heuristic:

closer to true cost

expands fewer nodes

more efficient A\*

Example: Manhattan distance

dominates tiles-out-of-place.

## 26. Best and Worst Admissible Heuristics

Best possible admissible heuristic:  $h^*(n) =$  true cost to goal.

This leads to: → perfect search → minimal expansions.

Worst admissible heuristic:  $h(n) = 0$ .

This reduces A\* to: → uniform-cost search.

## 27. Performance Impact of Heuristic Quality

Better heuristic → fewer node expansions → faster search.

Example behavior:

Poor heuristic: millions of expansions.

Good heuristic: hundreds.

Heuristic quality directly affects: time

memory

feasibility.

## 28. A\* vs Uniform Cost Search

Uniform cost:  $f(n) = g(n)$

A\*:  $f(n) = g(n) + h(n)$

Uniform cost:

explores outward evenly

may move away from goal

A\*:

biased toward goal

still considers cost so far

## 29. A\* vs Greedy Search

Greedy:  $f(n) = h(n)$

A\*:  $f(n) = g(n) + h(n)$

Greedy:

rushes toward goal

may take bad paths

A\*:

hedges bets

balances path cost + estimate

## 30. Practical Uses of A\*

You will likely see this on conceptual questions.

Used in:

GPS navigation

robotics path planning

video game AI movement

logistics and scheduling

airline resource planning

Reason:

optimal pathfinding

efficient computation

## 31. Contour Expansion Concept

Visualization idea:

Uniform cost:

expands circular cost contours.

A\*:

stretches contours toward goal.

Greedy:

shoots straight toward goal.

This explains:

behavior differences

performance gains.

## 32. Major A\* Exam Traps

Trap 1: Return goal immediately when seen. → wrong.

Trap 2: Admissible = exact. → false (can underestimate).

Trap 3: Greedy optimal. → false.

Trap 4: A\* always faster than greedy. → false (greedy may expand fewer nodes but is risky).

Trap 5: Admissible heuristic must be consistent. → false.

## 10. Consistent (Monotone) Heuristics

Stronger condition:

$h(n) \leq \text{cost}(n \rightarrow n') + h(n')$

This is triangle inequality.

Also requires:  $h(\text{goal}) = 0$

Implications:

heuristic decreases smoothly

A\* graph search remains optimal

no need to revisit nodes

Consistency  $\Rightarrow$  admissibility.

But: admissibility  $\neq$  consistency.

## 11. Heuristic Types

Three cases:

Consistent + admissible → ideal

Admissible but inconsistent → optimal but inefficient

Inadmissible → not optimal

These distinctions often appear on exams.

## 12. Uniform Cost, Greedy, and A\* Relationship

Key insight:

Uniform cost:  $f(n) = g(n)$

Greedy:  $f(n) = h(n)$

A\*:  $f(n) = g(n) + h(n)$

A\* generalizes both.

If  $h(n) = 0$ : → A\* becomes uniform-cost search.

If g(n) ignored: → becomes greedy search.

### 13. Shape of Search (Conceptual)

Slides illustrate:

BFS:

expands evenly outward.

UCS:

expands along cost contours.

A\*:

expands toward goal intelligently.

Greedy:

rushes toward goal.

This helps visualize search behavior.

### 14. Designing Good Heuristics

Good heuristics:

approximate real cost closely

cheap to compute

never overestimate

Sources:

relaxed problems

abstractions

domain knowledge

Better heuristic: → fewer nodes expanded.

### 15. Tradeoff: Accuracy vs Cost

More accurate heuristic:

reduces search work.

But:

may be expensive to compute.

Balance:

fast heuristic

informative guidance.

### 16. Complexity of Informed Search

Worst case:

still exponential.

But: good heuristic drastically reduces:

branching factor

nodes explored

Heuristic quality directly impacts performance.

### Module 4

#### 1. From “Puzzles” to “Games”: Why Adversarial Search Is Different

So far in search (BFS/DFS/UCS/A\*), you're usually solving a puzzle: the environment doesn't actively try to stop you.

Puzzle: Trying to find a solution, but not playing against a physical opponent

A game introduces an opponent whose objective conflicts with yours, so the “environment” becomes strategic (your outcome depends on your move and the opponent's response). The “simplest game environments” are often modeled as:

Multiagent, typically two-player, turn-taking, and zero-sum (one wins = other loses; draws possible).

Usually also: static, discrete, fully observable, and deterministic (for chess-like games).

Static: the agent can sit and think for as long as it needs them. The configuration of the game doesn't change while its thinking

Discrete: set of actions to take to transition from one state to another is finite

Fully observable: you see the complete configuration of the chessboard and you're not hiding things back from it

Strategic: Involves two or more agents who are against each other

Complicated: Like card games are usually not fully observable where you don't see the complete configuration of the game. Pacman

Why AI researchers like games (high-level motivation)

Games are clean “laboratories” for AI: State is easy to represent (board position).

Legal actions are crisp (rules).

But the search space is enormous (chess, Go). Your transcript's examples (Deep Blue, AlphaGo) are exactly the motivational story that leads into minimax + pruning.

#### 2. Core Modeling: Game as a Search Problem (Game Trees)

Game tree: is the search tree that follows from applying every sequence of moves all the way to all possible terminal tickets

In adversarial search, we build a game tree:

Nodes = game states (board positions)

Edges = legal moves

Root = current state

Terminal states = game-over (win/lose/draw)

Utility (a number) is defined for terminal outcomes (classic: win=+1, draw=0, loss=-1 for MAX)

Key idea: Unlike regular search where you just want a goal, here you want the move that yields the best outcome assuming the opponent responds well.

Terminology you'll be tested on

MAX: “us” (the player we're choosing a move for); tries to maximize the utility.

MIN: opponent; tries to minimize MAX's utility.

Ply: one move by one player = one level in the game tree (MAX ply then MIN ply then MAX ply...).

Adversarial search: where one or more agents (opponents) have conflicting goals. These types of algorithms anticipate the counter-moves of opponents to find the optimal strategy for maximizing one's own gain while minimizing the opponents.

#### 3. AIMA 5.1 — Game Theory (What you need for the midterm level)

AIMA's “game theory” framing is basically: in multi-agent settings, you

can think of agents as optimizing objectives that may be:

Fully opposed (zero-sum competitive)

Partly aligned (non-zero-sum → alliances, cooperation, negotiation)

Uncertain / mixed (randomness, hidden information, etc.)

For your midterm scope, the most important takeaway is:

The zero-sum, turn-taking, perfect-information model is the “starter case” where minimax applies cleanly.

When you move away from those assumptions (randomness / imperfect info / non-zero-sum), you need extensions (expectimax, etc.).

#### 4. AIMA 5.2 — Optimal Decisions in Games: The Minimax Idea

The minimax “rule”

Don't play hope chess. Meaning: don't pick a move just because it looks good if the opponent cooperates. Assume the opponent responds optimally.

Minimax value definition (conceptual)

Don't take a move that looks the best for you without thinking ahead at what your opponent will do.

At a terminal node: return its utility.

At a MAX node: value is the max of children's values.

At a MIN node: value is the min of children's values.

If max wins, it receives a positive score. If min wins, it receives a negative score as min tries to reduce its score while max tries to increase its score. Draw: 0 score for both parties

Weighted sum: is useful in determining how good the current player is doing based on his current number of positions and value of his pieces.

So minimax is a recursive backup procedure:

Leaf utilities get propagated up the tree by alternating max/min.

What minimax returns

At the root (MAX's turn), minimax returns:

the best achievable utility given optimal opponent play and therefore the argmax move to choose.

Complexity (why we need pruning)

If branching factor is  $b$  and depth is  $m$ , full minimax is:

Time:  $O(b^m)$  (exponential)

Space: depends on implementation; depth-first recursion is  $O(bm)$  or  $O(m)$  stack + storage choices.

This exponential blow-up is the reason alpha-beta exists.

#### 5. AIMA 5.2/5.3 — Alpha-Beta Pruning: Same Answer, Fewer Nodes

Alpha-beta pruning is minimax with a trick: it avoids exploring branches that